

Manual of the Field-Free VC-MTJ Compact Model

An experimentally calibrated probabilistic compact model with Verilog-A and Python implementations

Version: 1.1

Release Date: 2026/09/28

Yantong Di¹, Yaoru Hou², Xuezhao Wu², Hongjie Yin¹, Hao Cai¹ #, Qiming Shao² #

¹School of Integrated Circuits, Southeast University, Nanjing, China; ²Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, China;

Contact: hao.cai@seu.edu.cn, and eeqshao@ust.hk

Version	Release Date	Description of Changes	Contributor
1.0	2026/07/30	Initial release. 1. Developed a probabilistic compact model for the field-free VC-MTJ using Verilog-A. 2. Implemented the evaluation of voltage-dependent resistance and stochastic switching. 3. Provided typical and Monte Carlo simulation configurations in Virtuoso. 4. Included comprehensive netlist and schematic testbench examples.	Yantong, Yaoru, Xuezhao, Hao Cai, and Qiming
1.0.1	2026/09/16	Updated repository directory names to match the current GitHub package structure. No changes to the Verilog-A model behavior.	Yantong
1.1	2026/09/28	1. Added the Python implementation and corresponding usage documentation. 2. Reorganized the manual for the common model behavior and separate Verilog-A/Python implementations. 3. Updated package organization and user-accessible settings for both implementations. 4. Added Python reference results and Verilog-A/Python cross-implementation verification.	Yantong, Hongjie, Yaoru, Xuezhao, Hao Cai, and Qiming

Contents

I. General Introduction

II. Files Provided

II.A Repository Organization

II.B Verilog-A Package

II.C Python Package

III. Common Model Behavior

III.A State Representation

III.B Electrical and Switching Behavior

III.C Monte Carlo Behavior

IV. Verilog-A Implementation

IV.A Device Symbol and Pins

IV.B Initial State and Model Corners

IV.C Netlist-Based Typical Simulation

IV.D Netlist-Based Monte Carlo Simulation

IV.E Schematic-Based Typical Simulation

IV.F Schematic-Based Monte Carlo Simulation

V. Python Implementation

V.A Installation and Public Interface

V.B Basic Write/Read Simulation

V.C Device-Level Monte Carlo Simulation

V.D Pulse-Sequence Simulation

V.E Probability and Resistance Analysis

V.F Execution Backend and Usage Notes

VI. Reference Results and Implementation Verification

VI.A Verilog-A Reference Results

VI.B Python Reference Results

VI.C Cross-Implementation Verification

VII. User-Accessible Settings and Implementation Mapping

VIII. Usage Notes and Model Limitations

I. General Introduction

Voltage-controlled magnetic anisotropy magnetic tunnel junctions (VC-MTJs) provide a voltage-driven approach to controlling the magnetic state of an MTJ. The field-free VC-MTJ considered in this model is intended for the evaluation of voltage-dependent resistance and stochastic state switching without requiring an externally applied magnetic field.

The field-free VC-MTJ compact model is an experimentally calibrated probabilistic model describing the electrical response of the MTJ in its parallel (P) and antiparallel (AP) states and the probability of switching under an applied write pulse. The switching response depends on the voltage across the MTJ, the effective pulse duration, the current state, and the calibrated device parameters.

Two implementations are provided. The Verilog-A implementation is intended for Cadence Spectre and Virtuoso circuit simulation and supports typical and mismatch Monte Carlo analyses through netlist- and schematic-based testbenches. The Python implementation provides standalone nominal and device-level Monte Carlo simulation, stochastic single-pulse and pulse-sequence evaluation, and resistance, R-V, and TMR analysis.

Both implementations follow the same calibrated compact-model definitions but provide different user interfaces. The Verilog-A model obtains the device excitation from circuit terminal waveforms, whereas the Python model accepts write voltage, pulse width, state, and optional Monte Carlo populations directly through its API. The model is a probabilistic compact model rather than a full magnetization-dynamics solver; it does not numerically integrate the complete stochastic Landau-Lifshitz-Gilbert equation or provide a continuous magnetization trajectory.

II. Files Provided

II.A Repository Organization

```
VC-MTJ_MC_Model/  
├── 0_VC_MTJ_VerilogA_Model/  
├── 1_VC_MTJ_Python_Model/  
├── Manual_Field_Free_VC-MTJ_v1.1.pdf  
└── README.md
```

The release repository provides two implementations of the same field-free VC-MTJ compact model. The Verilog-A package is intended for Cadence Spectre/Virtuoso circuit simulation, while the Python package provides standalone statistical and stochastic simulation. The root README provides package-level navigation, and this manual describes the common model definitions and the usage of both implementations.

II.B Verilog-A Package

➤ Package organization

```
0_VC_MTJ_VerilogA_Model/  
├── README.md  
├── model_file/  
│   ├── vc_mtj_core.lib  
│   ├── vc_mtj_core.mdl  
│   └── vc_mtj_core.va  
└── testbench/  
    ├── schematic_based/  
    │   ├── README.md  
    │   ├── tb/  
    │   │   ├── schematic/  
    │   │   └── adexl/  
    │   └── vc_mtj_3t/  
    └── netlist_based/  
        ├── README.md  
        ├── tb_typical.scs  
        ├── run_tb_typical.sh  
        ├── tb_mc.scs  
        └── run_tb_mc.sh
```

➤ Model Files

File	Content
<i>model_file/vc_mtj_core.lib</i>	Defines model parameters and the mtj_t/ mtj_mc model corners, then includes the Spectre wrapper.

File	Content
<i>model_file/vc_mtj_core.mdl</i>	Defines the external three-terminal Spectre subcircuit vc_mtj_3t and maps model-library parameters to the Verilog-A module.
<i>model_file/vc_mtj_core.va</i>	Implements voltage-dependent resistance, probabilistic switching, internal random-number generation, and P/AP state updates.

Users normally include the .lib file and instantiate vc_mtj_3t, direct modification of the .mdl or .va files is not required for standard use.

➤ Testbench Files

File or Directory	Description
<i>testbench/netlist_based/tb_typical.scs</i>	Typical-corner transient example using section=mtj_t.
<i>testbench/netlist_based/run_tb_typical.sh</i>	Runs the typical netlist testbench and writes the typical simulation results.
<i>testbench/netlist_based/tb_mc.scs</i>	Mismatch Monte Carlo example using section=mtj_mc.
<i>testbench/netlist_based/run_tb_mc.sh</i>	Runs the Monte Carlo netlist testbench and writes the MC simulation results.
<i>testbench/schematic_based/vc_mtj_3t/</i>	Virtuoso device cell containing the three-terminal symbol and model interface.
<i>testbench/schematic_based/tb/schematic/</i>	Virtuoso schematic testbench for the VC-MTJ compact model.
<i>testbench/schematic_based/tb/adexl/</i>	Saved ADE XL setup supporting both typical transient and mismatch Monte Carlo simulations.
README.md files	Provide quick-start instructions for the package root and each testbench style.

Running the supplied netlist testbenches produces result directories and Spectre log files in *testbench/netlist_based/*, which are generated locally and are not required model-source files.

Testbench	Generated Output
Typical	<i>psf_typical/</i> and <i>spectre_typical.log</i>
Monte Carlo	<i>psf_mc/</i> and <i>spectre_mc.log</i>

II.C Python Package

➤ Package organization

```
1_VC_MTJ_Python_Model/
├── vcmtj/
│   ├── __init__.py
│   ├── model.py
│   ├── parameters.py
│   ├── _numpy.py
│   └── _numba.py
├── examples/
│   ├── 01_basic_simulation.py
│   ├── 02_monte_carlo.py
│   └── 03_sequence.py
├── tests/
│   └── test_model.py
├── README.md
└── pyproject.toml
```

➤ Model Files

File	Description
<i>vcmtj/__init__.py</i>	Package entry point and public exports.
<i>vcmtj/model.py</i>	Public API for VC-MTJ simulation.
<i>vcmtj/parameters.py</i>	Calibrated compact-model parameters.
<i>vcmtj/_numpy.py</i>	NumPy numerical implementation.
<i>vcmtj/_numba.py</i>	Optional Numba-accelerated simulation backend.

➤ Example and Support Files

File or Directory	Description
<i>examples/01_basic_simulation.py</i>	Nominal single-pulse write/read example.
<i>examples/02_monte_carlo.py</i>	Device-level Monte Carlo switching example.
<i>examples/03_sequence.py</i>	Multi-pulse stochastic sequence example.
<i>tests/test_model.py</i>	Regression and implementation-validation tests.
<i>pyproject.toml</i>	Package and dependency configuration.
<i>README.md</i>	Python-package quick-start information.

III. Common Model Behavior

III.A State Representation

The compact model represents the VC-MTJ using two discrete resistance states: P and AP. The P state corresponds to the lower-resistance state and the AP state to the higher-resistance state. Both the Verilog-A and Python implementations use the same P/AP state convention.

III.B Electrical and Switching Behavior

The P-state resistance, AP-state resistance, and tunnel magnetoresistance are voltage dependent; therefore, the VC-MTJ should not be treated as a fixed two-resistor element.

During a write operation, the switching probability is evaluated from the voltage across the MTJ, the effective pulse duration, the current P/AP state, and the calibrated device parameters. A stochastic decision then determines whether the state switches or remains unchanged.

The Verilog-A and Python implementations use the same calibrated resistance and switching behavior, while providing different simulation interfaces.

III.C Monte Carlo Behavior

The model supports device-to-device variation in addition to stochastic switching. In Monte Carlo simulation, different device realizations may exhibit different resistance and switching characteristics, while each write operation remains probabilistic. The Verilog-A and Python implementations provide different interfaces for enabling these variations, as described in Sections IV and V.

IV. Verilog-A Implementation

IV.A Device Symbol and Pins

The Verilog-A implementation is exposed to Spectre through the three-terminal subcircuit `vc_mjt_3t`. Its port order is `p`, `n`, `state`. The corresponding Verilog-A module is `VC_MJT_core(p, n, state)`.

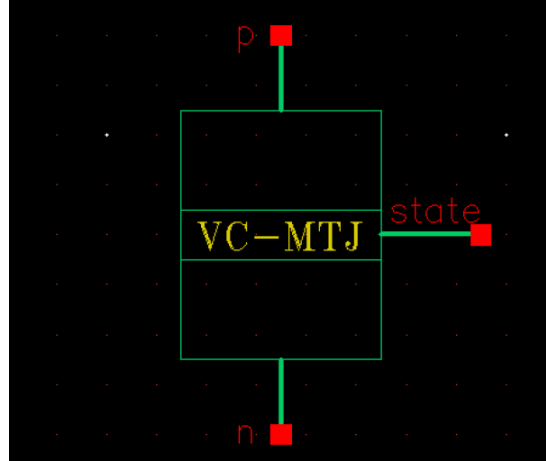


Fig. 1. Virtuoso symbol of the `vc_mjt_3t` compact model.

Pin	Type	Description
p	Analog electrical terminal	Positive reference terminal of the MTJ. The model defines $V_{MTJ} = V(p,n)$.
n	Analog electrical terminal	Negative reference terminal of the MTJ.
state	Observation output	Outputs the MTJ resistance state as a smoothed voltage: 0V for P and 1V for AP.

The state pin is provided for observing the internal MTJ state. It should not be driven by an external voltage source or connected as an electrical control input.

IV.B Initial State and Model Corners

➤ Initial State

The initial resistance state is selected using the instance parameter `Init`.

Setting	Type	Description
Init=0	Parallel (P)	0 V
Init=1	Antiparallel (AP)	1 V

For a Spectre netlist, the initial state is assigned in the instance statement:

```
I0 (VMTJ net1 State) vc_mjt_3t Init=1
```

In a Virtuoso schematic, the same parameter is set through the instance properties or the component CDF, depending on the final packaged cell configuration.

Cell Name	vc_mtj_3t	off
View Name	symbol	off
Instance Name	I2	off

Add Delete Modify

CDF Parameter	Value	Display
Initial State (0=P, 1=AP)	1	off

Fig. 2. Setting the Init parameter in Virtuoso.

➤ Model corners

Model Section	Description
mtj_t	Uses nominal model parameters for typical simulation.
mtj_mc	Enables statistical model parameters for mismatch Monte Carlo simulation.

The model corner is selected through the section argument when `vc_mtj_core.lib` is included. The `mtj_mc` corner is used together with mismatch Monte Carlo sampling, as described in Section IV.D.

IV.C Netlist-Based Typical Simulation

The typical example is provided in `0_VC_MTJ_VerilogA_Model/testbench/netlist_based/`. It uses the `mtj_t` model corner and performs a standard transient simulation.

➤ Model inclusion and instantiation

```
include "../model_file/vc_mtj_core.lib" section=mtj_t
I0 (VMTJ net1 State) vc_mtj_3t Init=1
```

In the provided example, the MTJ is initialized in the AP state and driven by a 2.4V pulse over a 10ns transient simulation. The complete stimulus and analysis settings are defined in `0_VC_MTJ_VerilogA_Model/testbench/netlist_based/tb_typical.scs`.

➤ Run procedure

1. Open a shell in the release package or change to the netlist-based testbench directory.
 2. Run the supplied script.
 3. Check `spectre_typical.log` for completion and open the generated PSF results in ViVA.
- ```
cd 0_VC_MTJ_VerilogA_Model/testbench/netlist_based
./run_tb_typical.sh
```

The script writes results to *psf\_typical/* and simulator output to *spectre\_typical.log*. The example is intended to verify model compilation, transient execution, and basic state/resistance behavior.

## IV.D Netlist-Based Monte Carlo Simulation

The Monte Carlo example is provided in the same directory. It uses the *mtj\_mc* model corner and performs a mismatch Monte Carlo simulation.

### ➤ Model inclusion and instantiation

```
include "../model_file/vc_mtj_core.lib" section=mtj_mc
I0 (VMTJ net1 State) vc_mtj_3t Init=1
```

### ➤ Monte Carlo analysis

```
mc1 montecarlo
numruns=1
seed=24680
variations=mismatch
donominal=no
{
 tran stop=10 ns
}
```

Both the *mtj\_mc* model corner and *variations=mismatch* are required to enable mismatch sampling. The complete settings are defined in *tb\_mc.scs*.

### ➤ Run procedure

1. Open a shell in the release package or change to the netlist-based testbench directory.
2. Run the supplied script.
3. Check *spectre\_mc.log* for completion and open the generated PSF results in ViVA.

```
cd 0_VC_MTJ_VerilogA_Model/testbench/netlist_based
./run_tb_mc.sh
```

The results are written to *psf\_mc/* and the simulator output to *spectre\_mc.log*.

## IV.E Schematic-Based Typical Simulation

The schematic-based package is located in *0\_VC\_MTJ\_VerilogA\_Model/testbench/schematic\_based/*. The saved *tb/adexl* view provides a unified ADE XL setup for both typical transient and mismatch MC simulations. The same *tb/schematic* testbench and saved output definitions are used in both cases.

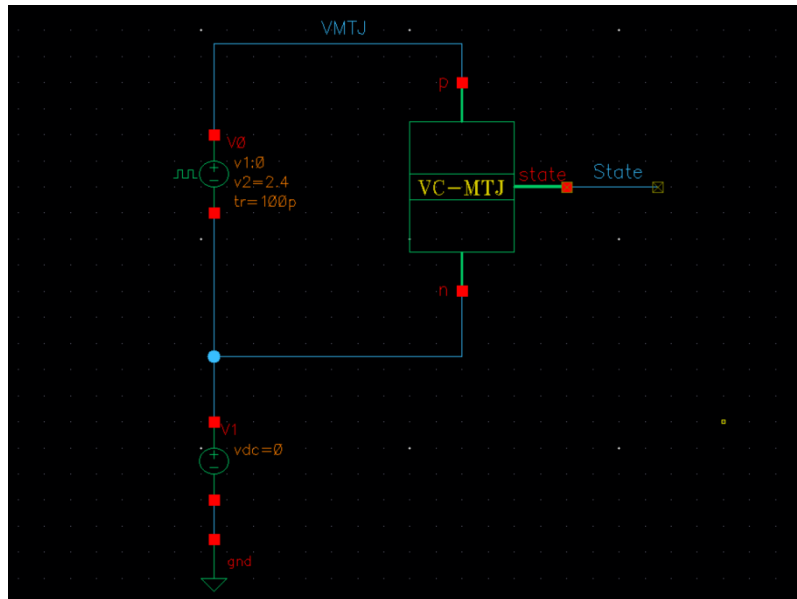


Fig. 3. Virtuoso schematic-based testbench for the VC-MTJ compact model.

### ➤ Opening the saved setup

1. Add *testbench/schematic\_based/* as a Virtuoso library and open the saved *tb/adexl* view.
2. Update the library path to the local location of *model\_file/vc\_mtl\_core.lib* when necessary.
3. Confirm that the configured transient analysis and the saved outputs are enabled.

### ➤ Typical transient run

1. Select **Single Run, Sweeps and Corners** as the run mode.
2. In **Corners Setup**, enable the typical corner, which uses the *mtj\_t* model section.
3. Run the configured transient analysis and inspect the saved waveforms in ViVA.

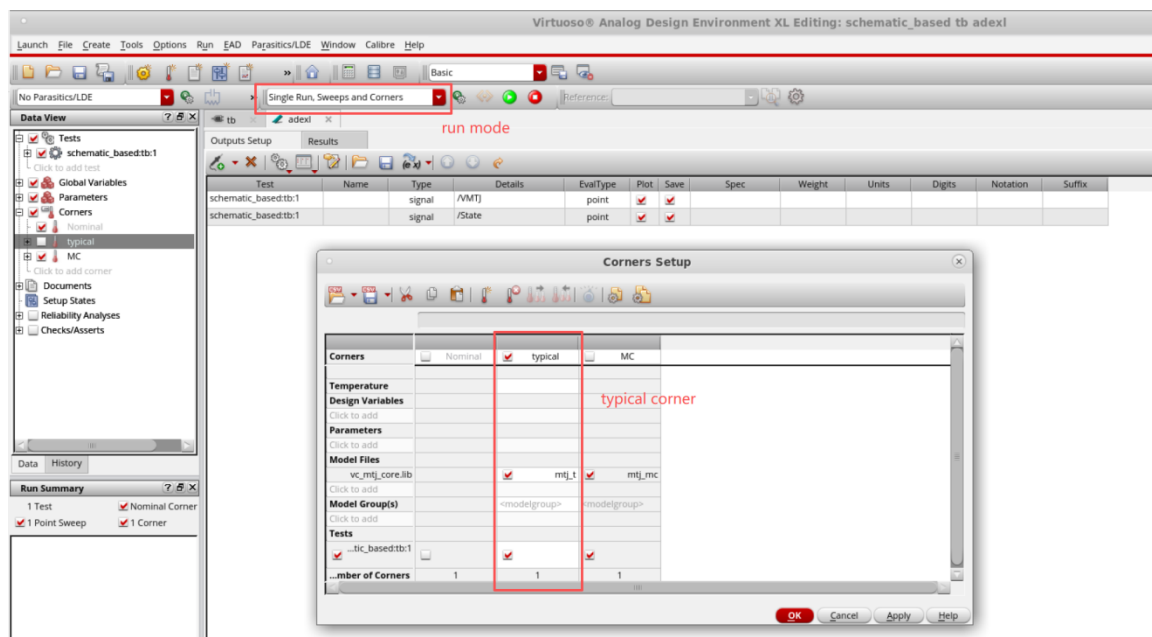


Fig. 4. ADE XL setup for the schematic-based typical transient simulation.

## IV.F Schematic-Based Monte Carlo Simulation

The same saved *tb/adexl* view is used for mismatch Monte Carlo simulation. The schematic, transient analysis, and output definitions remain unchanged; only the run mode, active corner, and Monte Carlo settings are changed.

### ➤ Monte Carlo run

1. Select Monte Carlo Sampling as the run mode.
2. In Corners Setup, enable the MC corner, which uses the *mtj\_mc* model section.
3. In the Monte Carlo settings, select Mismatch as the statistical variation. Set the required number of points and Monte Carlo seed before running the simulation.
4. Run the Monte Carlo simulation and inspect the saved results.

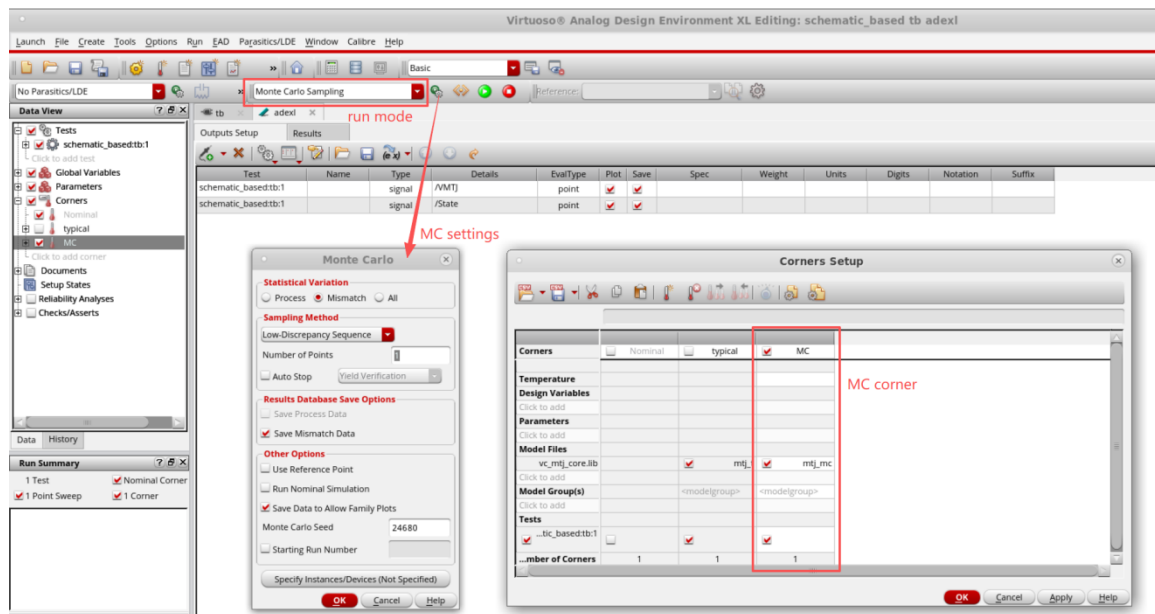


Fig. 5. ADE XL setup for the schematic-based mismatch Monte Carlo simulation.

## V. Python Implementation

### V.A Installation and Public Interface

The Python implementation is provided in *I\_VC\_MTJ\_Python\_Model/*. It requires Python 3.10 or later and NumPy 1.24 or later. Numba is optional and can be installed for accelerated execution.

#### ➤ Installation procedure

1. Open a terminal in the *I\_VC\_MTJ\_Python\_Model/* directory containing *pyproject.toml*.
2. Install the package using the following command:

```
python -m pip install .
```

To enable optional Numba acceleration, install the package with the numba extra:

```
python -m pip install "[numba]"
```

The package can run using NumPy alone. Backend behavior is described in Section V.F.

#### ➤ Public interface

The main simulation interface and supporting objects are imported from *vcmtj*:

```
from vcmtj import MTJModel, DevicePopulation, P, AP
```

The main public objects are summarized below.

| Object           | Description                                                                                                                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MTJModel         | Provides the main simulation interface through <code>simulate()</code> , together with <code>evaluate_psw()</code> for switching-probability evaluation and <code>resistance()</code> for resistance and TMR evaluation. |
| DevicePopulation | Represents a fixed set of nominal or Monte Carlo device realizations. Device-level variations remain fixed throughout a pulse sequence.                                                                                  |
| P / AP           | State constants representing the parallel and antiparallel states, respectively: $P = 0$ and $AP = 1$ .                                                                                                                  |
| SimulationResult | Returned by <code>simulate()</code> . Stores switching probabilities, switching outcomes, state trajectories, and optional resistance and TMR results.                                                                   |
| ResistanceResult | Returned by <code>resistance()</code> . Stores resistance and TMR results.                                                                                                                                               |

Examples of basic simulation, Monte Carlo simulation, and pulse-sequence simulation are provided in Sections V.B–V.D.

## V.B Basic Write/Read Simulation

The basic example is provided in `1_VC_MTJ_Python_Model/examples/01_basic_simulation.py`. It applies a single write pulse to one nominal device initialized in the AP state and evaluates the resistance and TMR at a specified read voltage.

### ➤ Model initialization and simulation

The following Python code creates the model and performs a single-pulse simulation:

```
from vcmtj import MTJModel, AP
model = MTJModel()
run = model.simulate(
 write_voltage_v=1.6,
 pulse_width_ns=1.2,
 initial_state=AP,
 switching_seed=456,
 read_voltage_v=0.1,
)
```

In this example, the actual MTJ write voltage is 1.6 V and the effective write pulse width is 1.2 ns. Since no device population is supplied, the simulation uses one nominal device without device-to-device variation. Switching remains stochastic.

The input settings are summarized below.

| Input           | Description                                                                                                                           |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------|
| write_voltage_v | Actual write voltage across the MTJ terminals, in volts.                                                                              |
| pulse_width_ns  | Effective write pulse-top duration, in nanoseconds.                                                                                   |
| initial_state   | Initial MTJ state, specified as P or AP.                                                                                              |
| switching_seed  | Seed for stochastic switching during this simulation call. Using the same seed with identical inputs reproduces the switching result. |
| read_voltage_v  | Optional read voltage, in volts, used to evaluate resistance and TMR for stored states.                                               |

The read voltage is used for resistance and TMR evaluation and does not apply an additional switching pulse. If `read_voltage_v` is omitted, the resistance and TMR results are not calculated.

### ➤ Simulation outputs

The returned object `run` stores the following results.

| Output                         | Description                                                                              |
|--------------------------------|------------------------------------------------------------------------------------------|
| <code>run.probabilities</code> | Switching probability for each write pulse, evaluated using the state before that pulse. |

| Output             | Description                                                                                                                       |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| run.switched       | Boolean switching outcome for each write pulse: True if the state switches and False otherwise.                                   |
| run.states         | MTJ states, including the initial state and the state after each write pulse. The state convention is $P = 0$ and $AP = 1$ .      |
| run.resistance_ohm | Resistance corresponding to each stored state at the specified read voltage, in ohms. Set to None if no read voltage is supplied. |
| run.tmr            | TMR at the specified read voltage, expressed as a dimensionless ratio. Set to None if no read voltage is supplied.                |

For this single-device, single-pulse example, the results can be displayed using:

```
print("Psw:", run.probabilities[0, 0])
print("switched:", run.switched[0, 0])
print("final state:", run.states[-1, 0])
print("final resistance (ohm):", run.resistance_ohm[-1, 0])
print("TMR:", run.tmr[0])
```

Here, *[0, 0]* selects the first pulse and the first device, while *run.states[-1, 0]* selects the final state of that device.

#### ➤ Run procedure

1. Open a terminal in the *I\_VC\_MTJ\_Python\_Model/* directory after completing the installation in Section V.A.

2. Run the supplied example:

```
python examples/01_basic_simulation.py
```

3. Inspect the switching probability, switching outcome, final state, resistance, and TMR printed in the terminal.

Reference results are provided in Section VI.B.

## V.C Device-Level Monte Carlo Simulation

The Monte Carlo example is provided in *I\_VC\_MTJ\_Python\_Model/examples/02\_monte\_carlo.py*. It applies a single write pulse to a population of 10,000 devices, including device-to-device variation and stochastic switching.

#### ➤ Device population and simulation

First, create the model and a fixed Monte Carlo device population:

```
import numpy as np
from vcmtj import MTJModel, DevicePopulation, AP
```

```
model = MTJModel()
```

```
devices = DevicePopulation.monte_carlo(
 n_devices=10_000,
 device_seed=123,
)
```

Then, pass the population to *simulate()*:

```
run = model.simulate(
 write_voltage_v=1.6,
 pulse_width_ns=1.2,
 initial_state=AP,
 population=devices,
 switching_seed=456,
 read_voltage_v=0.1,
)
```

All devices are initialized in the AP state and receive the same 1.6 V write pulse with an effective duration of 1.2 ns. Each device retains its own sampled characteristics, which may produce different switching probabilities and resistance values.

The population and seed settings are summarized below. The remaining simulation inputs follow the definitions in Section V.B.

| Setting        | Description                                                                                                                   |
|----------------|-------------------------------------------------------------------------------------------------------------------------------|
| n_devices      | Number of device realizations in the population.                                                                              |
| device_seed    | Seed used to generate the device population. Using the same seed and population size reproduces the same device realizations. |
| population     | Device population supplied to the simulation.                                                                                 |
| switching_seed | Seed used to generate the random numbers for stochastic switching during the simulation call.                                 |

The two seeds control different sources of randomness. *device\_seed* determines the fixed device realizations, whereas *switching\_seed* controls the stochastic switching trials. Changing only *switching\_seed* keeps the device characteristics unchanged while generating a new set of switching trials.

The variations stored in a *DevicePopulation* remain fixed throughout a pulse sequence. Reusing the same population in another simulation also preserves those device characteristics.

### ➤ Simulation outputs

The returned fields follow Section V.B and contain results for all devices. Population-level

statistics can be displayed using:

```
print("mean Psw:", run.probabilities[0].mean())
print("switched fraction:", run.switched[0].mean())
print("final AP fraction:", np.mean(run.states[-1] == AP))
print(
 "mean final resistance (ohm):",
 run.resistance_ohm[-1].mean(),
)
```

The mean switching probability is the average of the calculated device probabilities, while the switched fraction is the fraction of devices that actually switch in this simulation. These quantities need not be identical because the switching outcomes are stochastic. Since all devices start in the AP state and receive one pulse, the final AP fraction equals one minus the switched fraction.

#### ➤ **Run procedure**

1. Open a terminal in the *I\_VC\_MTJ\_Python\_Model/* directory after completing the installation in Section V.A.

2. Run the supplied example:

```
python examples/02_monte_carlo.py
```

3. Inspect the mean switching probability, switched fraction, final AP fraction, and mean final resistance printed in the terminal.

Reference results are provided in Section VI.B.

## **V.D Pulse-Sequence Simulation**

The pulse-sequence example is provided in *I\_VC\_MTJ\_Python\_Model/examples/03\_sequence.py*. It applies three consecutive write pulses to eight Monte Carlo devices and records their switching outcomes and state trajectories.

#### ➤ **Pulse-sequence setup**

The following code creates a fixed device population and applies a shared pulse sequence:

```
from vcmtj import MTJModel, DevicePopulation, AP
```

```
model = MTJModel()
```

```
devices = DevicePopulation.monte_carlo(
 n_devices=8,
 device_seed=123,
)
```

```
run = model.simulate(
 write_voltage_v=[1.6, 1.7, 1.6],
```

```

pulse_width_ns=[1.0, 1.1, 1.2],
initial_state=AP,
population=devices,
switching_seed=456,
read_voltage_v=0.1,
)

```

When both write voltage and pulse width are scalars, the simulation applies one pulse. One-dimensional lists or arrays specify a pulse sequence shared by all devices. Corresponding entries in the voltage and pulse-width lists define each pulse.

In this example, the three pulses are 1.6 V for 1.0 ns, 1.7 V for 1.1 ns, and 1.6 V for 1.2 ns. All devices start in the AP state. After each pulse, the updated state is used to evaluate the switching probability for the next pulse. Device-level variations remain fixed throughout the sequence.

### ➤ Simulation outputs

Let  $N_p$  denote the number of pulses and  $N_d$  the number of devices. The output arrays are organized as follows.

| Output             | Shape                 | Description                                                                                   |
|--------------------|-----------------------|-----------------------------------------------------------------------------------------------|
| run.proBABILITIES  | ( $N_p$ , $N_d$ )     | Switching probability for each pulse and device, evaluated using the state before that pulse. |
| run.switched       | ( $N_p$ , $N_d$ )     | Boolean switching outcome for each pulse and device.                                          |
| run.states         | ( $N_p + 1$ , $N_d$ ) | Initial states followed by the states after each pulse.                                       |
| run.resistance_ohm | ( $N_p + 1$ , $N_d$ ) | Resistance at the specified read voltage, aligned with each row of run.states.                |
| run.tmr            | ( $N_d$ ,)            | TMR for each device at the specified read voltage.                                            |

The resistance and TMR outputs are available when *read\_voltage\_v* is supplied; otherwise, both fields are None.

Each column corresponds to one device. *run.states[0]* contains the initial states, *run.states[1]* contains the states after the first pulse, and *run.states[-1]* contains the final states after the complete sequence.

For this example, the probability and switching-outcome arrays have shape (3, 8), while the state and resistance arrays have shape (4, 8). The results can be displayed using:

```

print("probabilities (pulse x device):")
print(run.proBABILITIES)
print("switched (pulse x device):")
print(run.switched)

```

```

print("states (initial row followed by each pulse):")
print(run.states)
print("resistance aligned with each state row (ohm):")
print(run.resistance_ohm)

```

#### ➤ **Run procedure**

1. Open a terminal in the *I\_VC\_MTJ\_Python\_Model/* directory after completing the installation in Section V.A.

2. Run the supplied example:

```
python examples/03_sequence.py
```

3. Inspect the probability, switching-outcome, state, and resistance arrays printed in the terminal.

Reference results are provided in Section VI.B.

## **V.E Probability and Resistance Analysis**

The Python implementation provides two auxiliary methods for direct model analysis: *evaluate\_psw()* for switching-probability evaluation and *resistance()* for static resistance and TMR evaluation. Both methods can be used independently of a stochastic write simulation.

#### ➤ **Switching-probability analysis**

The *evaluate\_psw()* method calculates the switching probability for specified write conditions and a specified initial state. It supports probability-versus-voltage and probability-versus-pulse-width analysis.

```

from vcmtj import MTJModel, P, AP
model = MTJModel()
psw_v = model.evaluate_psw(
 write_voltage_v=[1.4, 1.6, 1.8],
 pulse_width_ns=1.2,
 initial_state=AP,
)
psw_pw = model.evaluate_psw(
 write_voltage_v=1.6,
 pulse_width_ns=[0.8, 1.0, 1.2, 1.4],
 initial_state=AP,
)
print("Psw versus write voltage:", psw_v)
print("Psw versus pulse width:", psw_pw)

```

The first call evaluates the switching probability at three actual MTJ terminal voltages with a fixed pulse width of 1.2 ns. The second call evaluates four pulse widths at a fixed write voltage of 1.6 V.

With *initial\_state=AP*, the returned values represent AP-to-P switching probabilities. Setting *initial\_state=P* evaluates P-to-AP switching probabilities.

Each sweep point is evaluated at the specified state. Unlike a pulse-sequence simulation, this method performs no stochastic switching trial or state update. It returns a NumPy array of probabilities and does not calculate resistance.

Calling *evaluate\_psw()* is not required before *simulate()*, which evaluates switching probabilities internally.

### ➤ Resistance and TMR analysis

The *resistance()* method evaluates resistance and TMR at a specified read voltage and MTJ state. It can be used to obtain P-state and AP-state resistances, resistance-versus-voltage characteristics, and voltage-dependent TMR.

```
read_voltages = [0.0, 0.1, 0.2]
result_p = model.resistance(
 read_voltage_v=read_voltages,
 mtj_state=P,
)
result_ap = model.resistance(
 read_voltage_v=read_voltages,
 mtj_state=AP,
)
print("P-state resistance (ohm):", result_p.resistance_ohm)
print("AP-state resistance (ohm):", result_ap.resistance_ohm)
print("TMR:", result_p.tmr)
```

The supplied voltages are the actual voltages across the MTJ terminals. The returned ResistanceResult object contains the following fields.

| Output         | Description                                                                             |
|----------------|-----------------------------------------------------------------------------------------|
| resistance_ohm | Resistance for the specified P or AP state at each read voltage, in ohms.               |
| tmr            | TMR at each read voltage, expressed as the dimensionless ratio $(R_{AP} - R_P) / R_P$ . |

The TMR is a property of the device at the specified voltage and is the same for P-state and AP-state evaluations of that device. A returned TMR of 0.5 corresponds to 50%.

The *resistance()* method can be called directly without first running *simulate()* or *evaluate\_psw()*. It evaluates static electrical characteristics without performing stochastic switching or changing the MTJ state.

➤ **Device-population analysis**

Both methods accept an optional *population* argument. Omitting this argument evaluates nominal device characteristics. Supplying a *DevicePopulation*, as described in Section V.C, evaluates the same quantities for the fixed device realizations without resampling their variations.

## **V.F Execution Backend and Usage Notes**

➤ **Execution backend**

NumPy provides the baseline implementation. When Numba is installed and available, *simulate()* automatically uses the accelerated backend for stateful pulse simulation. Users do not need to select a backend manually. The auxiliary methods *evaluate\_psw()* and *resistance()* use NumPy for vectorized analysis.

➤ **Usage notes**

Performance depends on the workload, including the number of devices and pulses. The first Numba call may incur compilation overhead; subsequent calls can reuse the compiled code.

Detailed model validity limits are summarized in Section VIII.

## VI. Reference Results and Implementation Verification

This section provides reference results for confirming that the released Verilog-A and Python implementations are installed and operating as expected. The stochastic switching sequence may vary with the random sequence, device realization, and simulation seed. Therefore, stochastic state trajectories should be interpreted as representative examples rather than required pulse-by-pulse results. Cross-implementation comparisons in Section VI.C focus on deterministic model quantities under matched nominal conditions rather than identical random switching sequences.

### VI.A Verilog-A Reference Results

#### ➤ Typical Switching Example

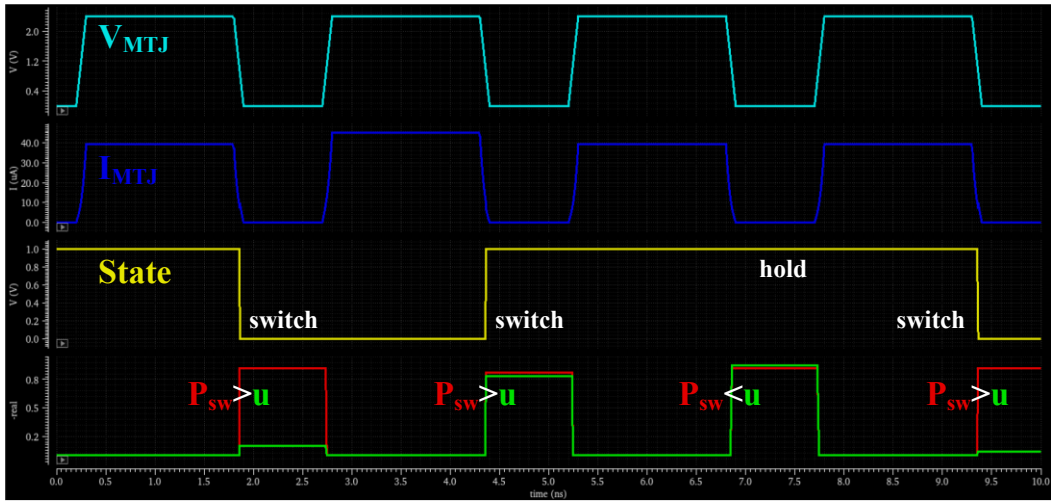


Fig. 6. Typical-corner transient response of the VC-MTJ compact model.

Fig. 6 shows the applied MTJ voltage  $V_{MTJ}$ , MTJ current  $I_{MTJ}$ , state output, switching probability  $P_{sw}$ , and pseudo-random number  $u$ . For each write pulse, the MTJ state switches when  $u < P_{sw}$  and remains unchanged otherwise. The MTJ current varies accordingly with the P/AP state.

#### ➤ Monte Carlo Example

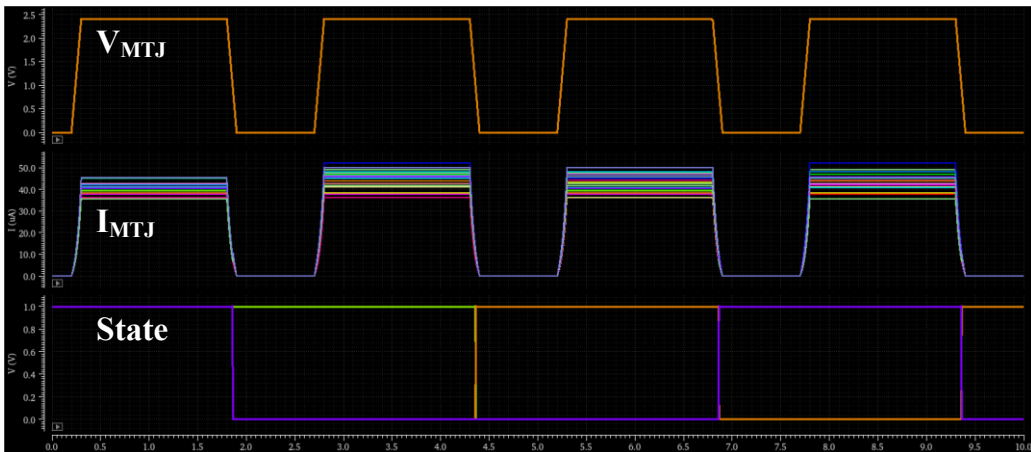


Fig. 7. Monte Carlo transient responses of the VC-MTJ compact model.

Each Monte Carlo run represents a different device realization. Under identical input pulses, variations are observed in the MTJ current and probabilistic switching sequence, as shown in Fig. 7.

### ➤ Voltage-Dependent Resistance

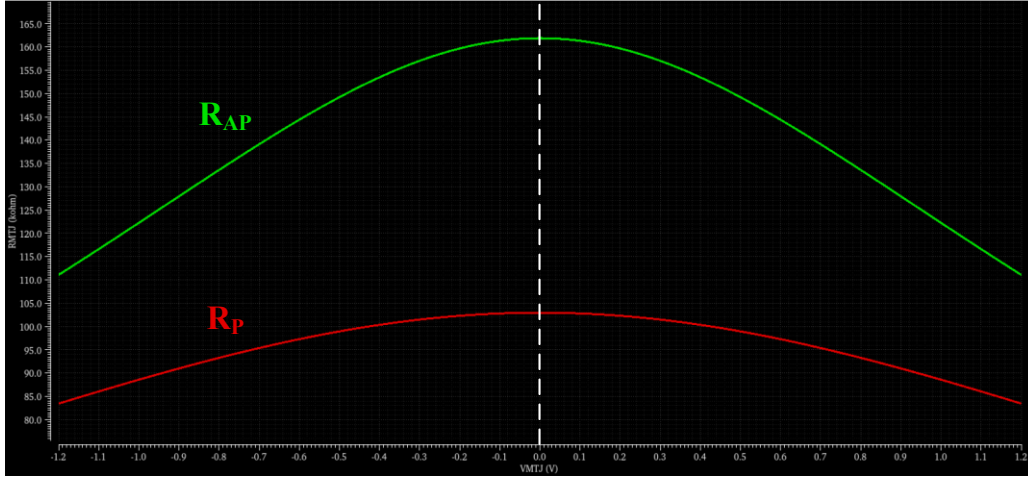


Fig. 8. Voltage-dependent P- and AP-state resistances of the VC-MTJ compact model.

The DC sweep in Fig. 8 illustrates the voltage-dependent resistance behavior implemented in the compact model. Both  $R_P$  and  $R_{AP}$  decrease as the magnitude of the applied MTJ voltage increases.

## VI.B Python Reference Results

The following results were obtained from the three supplied Python examples using Python 3.12.14, NumPy 2.3.5, and the NumPy backend. Numerical values are rounded for presentation. These results provide reference outputs for checking basic simulation, device-level Monte Carlo simulation, and pulse-sequence execution.

### ➤ Basic write/read example

The *01\_basic\_simulation.py* example applies a 1.6 V, 1.2 ns write pulse to one nominal device initialized in the AP state. The switching seed is 456, and the read voltage is 0.1 V.

| Quantity              | Reference result    |
|-----------------------|---------------------|
| Switching probability | 0.297716            |
| Switching outcome     | False               |
| Final state           | AP (1)              |
| Final resistance      | 161,312.85 $\Omega$ |
| TMR                   | 0.569322 (56.9322%) |

For this seed, the device remains in the AP state after the write pulse. The switching probability specifies the likelihood of switching; a single trial returns one Boolean outcome.

➤ **Device-level Monte Carlo example**

The `02_monte_carlo.py` example applies the same write and read conditions to 10,000 devices initialized in the AP state. The device seed is 123, and the switching seed is 456.

| Quantity                   | Reference result    |
|----------------------------|---------------------|
| Number of devices          | 10,000              |
| Mean switching probability | 0.313925            |
| Switched fraction          | 0.3116              |
| Final AP fraction          | 0.6884              |
| Mean final resistance      | 143,069.65 $\Omega$ |

A total of 3,116 devices switch from AP to P, while 6,884 remain in AP. The switched fraction is close to the mean switching probability, with a difference arising from finite stochastic sampling. The mean final resistance includes both P-state and AP-state devices and their sampled device variations.

➤ **Pulse-sequence example**

The `03_sequence.py` example applies 3 pulses to 8 Monte Carlo devices. The write voltages are  $[1.6, 1.7, 1.6]$  V, and the corresponding pulse widths are  $[1.0, 1.1, 1.2]$  ns. All devices start in AP. The device seed is 123, the switching seed is 456, and the read voltage is 0.1 V.

The table below summarizes the returned state trajectory. Device columns follow the order in `run.states`, with 0 representing P and 1 representing AP.

| Stage         | Device states   | Devices switched in this pulse | AP fraction |
|---------------|-----------------|--------------------------------|-------------|
| Initial       | 1 1 1 1 1 1 1 1 | —                              | 1.000       |
| After pulse 1 | 1 1 0 1 1 1 1 1 | 1                              | 0.875       |
| After pulse 2 | 1 1 0 1 0 1 1 1 | 1                              | 0.750       |
| After pulse 3 | 1 1 0 0 0 1 1 0 | 2                              | 0.500       |

The probability and switching-outcome arrays have shape  $(3, 8)$ . The state and resistance arrays have shape  $(4, 8)$ , including the initial row. Resistance values are evaluated at the specified read voltage for every stored state.

These stochastic outcomes correspond to the stated seeds and settings. Changing the seeds may change the sampled device characteristics or switching outcomes.

## VI.C Cross-Implementation Verification

Cross-implementation verification compares deterministic model quantities under matched conditions: P-state resistance, AP-state resistance, TMR, and switching probability. These comparisons assess whether the Python and Verilog-A implementations reproduce the same calibrated model behavior.

### ➤ Comparison conditions

The comparisons use nominal device parameters without mismatch variation. The Verilog-A implementation uses the `mtj_t` model corner, while the Python implementation uses `MTJModel()` without a Monte Carlo population.

The actual voltage across the MTJ terminals must match between implementations. For switching-probability comparisons, the effective pulse-top duration and the state before the pulse must also match. A programmed source voltage should not be substituted for the MTJ terminal voltage when circuit loading causes a voltage difference.

### ➤ Resistance consistency

The Python reference values below are calculated using `resistance()` for the P and AP states. The corresponding Verilog-A values were obtained at the same terminal voltages with the device in the specified state.

| Quantity                         | MTJ terminal voltage | Verilog-A    | Python       | Absolute difference   |
|----------------------------------|----------------------|--------------|--------------|-----------------------|
| P-state resistance ( $\Omega$ )  | 0.1 V                | 102,791.5090 | 102,791.4335 | 0.0755                |
| AP-state resistance ( $\Omega$ ) | 0.1 V                | 161,313.0625 | 161,312.8499 | 0.2126                |
| TMR                              | 0.1 V                | 0.569322837  | 0.569321921  | $9.16 \times 10^{-7}$ |
| P-state resistance ( $\Omega$ )  | 0.2 V                | 102,293.8893 | 102,293.7990 | 0.0903                |
| AP-state resistance ( $\Omega$ ) | 0.2 V                | 159,666.7436 | 159,666.5765 | 0.1671                |
| TMR                              | 0.2 V                | 0.560862967  | 0.5608627117 | $2.55 \times 10^{-7}$ |

Resistance is expressed in ohms, and TMR is dimensionless.

### ➤ Switching-probability consistency

The Python reference values below are calculated using `evaluate_psw()`.

The Verilog-A comparison were obtained from the switching probability evaluated for the corresponding pulse before the stochastic switching decision.

| Direction          | MTJ write voltage | Pulse width | Verilog-A  | Python      | Absolute difference    |
|--------------------|-------------------|-------------|------------|-------------|------------------------|
| AP $\rightarrow$ P | 1.6 V             | 1.2 ns      | 0.29771381 | 0.297716152 | $2.342 \times 10^{-6}$ |
| P $\rightarrow$ AP | 1.6 V             | 1.2 ns      | 0.25058745 | 0.250589475 | $2.025 \times 10^{-6}$ |
| AP $\rightarrow$ P | 1.7 V             | 1.1 ns      | 0.22856028 | 0.228562087 | $1.807 \times 10^{-6}$ |
| P $\rightarrow$ AP | 1.7 V             | 1.1 ns      | 0.19087978 | 0.190881328 | $1.548 \times 10^{-6}$ |

For each quantity, the absolute difference is calculated as  $abs(Python - Verilog-A)$  using the unrounded outputs.

➤ **Verification status and interpretation**

The Verilog-A and Python implementations show close agreement under matched nominal conditions. For resistance verification, the absolute differences in the P- and AP-state resistances remain below  $0.22 \, \Omega$  over the evaluated terminal voltages, while the corresponding TMR differences are below  $1 \times 10^{-6}$ . For switching-probability verification, the absolute differences between the two implementations are on the order of  $10^{-6}$  for all evaluated operating points and switching directions. These results demonstrate close agreement between the two implementations at the evaluated operating points. The remaining differences are sufficiently small and are consistent with implementation-level numerical and result-extraction differences.

## VII. User-Accessible Settings and Implementation Mapping

The Verilog-A and Python implementations follow the same calibrated compact-model behavior but expose different user interfaces. The table below summarizes the main user-accessible settings and their correspondence between the two implementations. Internal calibrated model coefficients are not intended for routine modification.

| Model concept            | Verilog-A implementation                                     | Python implementation                                                                        | Description                                                   |
|--------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Initial state            | $Init=0/1$                                                   | <code>initial_state=P/AP</code>                                                              | Selects the initial P or AP state.                            |
| Write voltage            | MTJ terminal waveform<br>$V(p,n)$                            | <code>write_voltage_v</code>                                                                 | Actual voltage across the MTJ during writing.                 |
| Pulse width              | Extracted from the MTJ terminal waveform                     | <code>pulse_width_ns</code>                                                                  | Effective write-pulse duration used for switching evaluation. |
| Nominal simulation       | <code>section=mtj_t</code>                                   | Default <code>MTJModel()</code> without a Monte Carlo population                             | Uses nominal calibrated device parameters.                    |
| Device-level Monte Carlo | <code>section=mtj_mc</code> with mismatch sampling           | <code>DevicePopulation.monte_carlo()</code> passed through <code>population</code>           | Enables device-to-device parameter variation.                 |
| Device realization seed  | Simulator Monte Carlo seed                                   | <code>device_seed</code>                                                                     | Controls the generated device realization or population.      |
| Switching randomness     | Internal model random-number generation                      | <code>switching_seed</code>                                                                  | Controls stochastic switching decisions.                      |
| Resistance evaluation    | Determined continuously from terminal voltage and P/AP state | <code>read_voltage_v</code> in <code>simulate()</code> or <code>model.resistance(...)</code> | Evaluates voltage-dependent resistance and TMR.               |

Unlike the Python interface, the Verilog-A package does not expose device variation and stochastic switching as two fully independent user-controlled seeds.

In the Verilog-A implementation, the write excitation is obtained directly from the circuit waveform across the MTJ terminals. In the Python implementation, the corresponding write voltage and pulse width are supplied explicitly through the API. The two interfaces therefore differ in how the excitation is provided, while using the same underlying calibrated resistance and switching behavior.

Calibrated model coefficients are stored in `model_file/vc_mtl_core.lib` for the Verilog-A implementation and in `vcmtj/parameters.py` for the Python implementation. These coefficients should not be modified independently without model recalibration.

## VIII. Usage Notes and Model Limitations

- The model is an experimentally calibrated probabilistic compact model and does not solve the full stochastic LLG equation. It is intended for circuit-level simulation through the Verilog-A implementation and standalone statistical simulation through the Python implementation.
- The switching response is calibrated over an internal fitted voltage range of 1.6–2.4V. Results outside this range should not be interpreted as independently validated physical predictions.
- The calibrated switching model requires an effective pulse-top duration of at least 0.7ns.
- The switching response is evaluated using the actual voltage across the MTJ terminals. In circuit simulations, this voltage may differ from the programmed source voltage because of access devices, interconnect resistance, or other loading. The corresponding actual MTJ voltage should therefore be used when providing write conditions to the Python implementation.
- The calibrated resistance and switching parameters should not be modified independently without model recalibration. The supplied Verilog-A testbenches and Python examples are functional usage examples; application-specific pulse conditions, read margins, and operating conditions should be verified separately.